Name: Arne Lieten

Email: Arne.Lieten@vub.be

Student Number: 0642644

VRIJE UNIVERSITEIT BRUSSEL

Pedigree Dog App: Pre Study

Table of Contents

Introduction **1**
Scope and architecture **1**
Types **2**
Server ADTs **3**
DogProfile 3
DatabaseHandler 3
ProcessOrchestrator 3
Client ADTs **5**
ClientOrchestrator 5
MainWindow 5
Testing **5**

Introduction

This pre-study outlines the phase 1 design of a pedigree dog matching application. The idea is to match pedigree dogs based on their age, hobby and type of relation. Later in the project more things can be taken into account such as their family tree, location, among other factors. This pre-study describes the main ADTs and their dependencies, as well as some pointers on testing.

Scope and architecture

Owners create profiles for their pedigree dogs. Phase 1 provides at least four independent GUIs and one shared server that stores profiles in memory. Each profile contains a name, age, hobbies and type of relation. Additional fields are left out for now. The application architecture is shown in Image 1. The application has a clear separation between client components (blue) and server components (yellow), with DogProfile shown in purple.

The server has one DatabaseHandler that provides operations to access the in-memory database, one ProcessOrchestrator that handles the server-side logic and DogProfile instances representing individual dogs' profiles. DatabaseHandler uses DogProfile's public operations to access profile data and does not need to know about ProcessOrchestrator. ProcessOrchestrator depends on DatabaseHandler to store data and on DogProfile to create profile instances.

Each client has a ClientOrchestrator that handles client-side logic and a MainWindow that manages its GUI. MainWindow handles opening and closing the window, drawing the profile form and displaying submission feedback using the `racket/gui/base` library. It passes submitted field values to ClientOrchestrator through a registered callback. MainWindow is responsible only for the GUI, while client-side processing remains in ClientOrchestrator, so MainWindow does not need to know about the ClientOrchestrator. In later phases, MainWindow will manage navigation between different widgets,

Name: Arne Lieten
Email: Arne.Lieten@vub.be
Student Number: 0642644

with each widget getting their own ADT. In phase 1 we keep it simple and just combine everything in MainWindow.

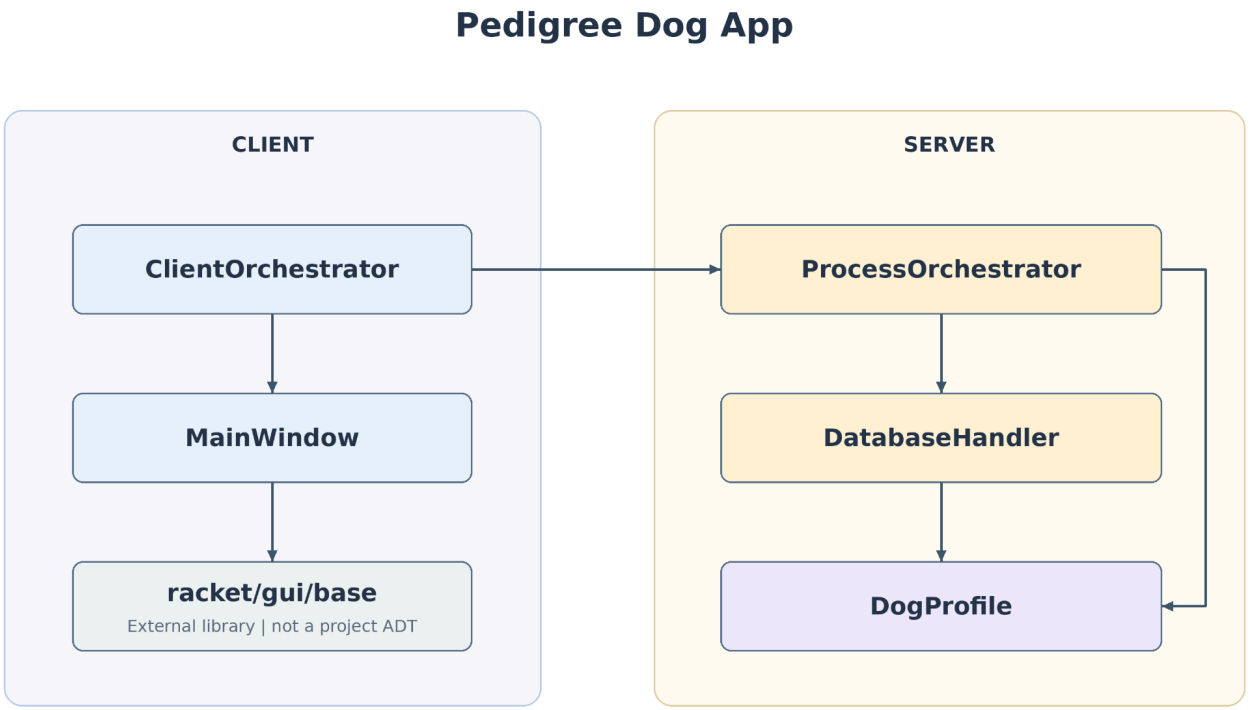


Image 1: ADT Dependency Diagram of Pedigree Dogs App

Types

Description of all the types in the operation signatures for the Server and Client ADTs are displayed in Table 1.

Type	Definition
ID	Unique identifier linked to a specific dog profile, created by the ProcessOrchestrator.
RawFields	Submitted values via the GUI. name (string), age (string), hobbies (listof string) and relation (string).
ProfileFields	Identical to RawFields, except for the age which is converted to an integer.
SubmitCallback	Procedure with signature RawFields -> void, called by MainWindow when the user submits the form.

Table 1: Types used in Server and Client ADTs

Name: Arne Lieten

Email: Arne.Lieten@vub.be

Student Number: 0642644

VRIJE UNIVERSITEIT BRUSSEL

Server ADTs

DogProfile

DogProfile represents information about a particular dog: id, name, age, hobbies and type of relation the dog is looking for. Table 2 gives the available operations and signatures. The main purpose is to initialize the DogProfile, when the fields are not valid the response should raise an exception.

Operation	Signature
initialize-dog-profile	ID, string, integer, (listof string), string -> DogProfile
profile-id	DogProfile -> ID
profile-name	DogProfile -> string
profile-age	DogProfile -> integer
profile-hobbies	DogProfile -> listof string
profile-relation	DogProfile -> string

Table 2: ADT of the DogProfile

DatabaseHandler

ADT responsible for the initialization of the InMemoryDatabase (not shown on architecture) in phase 1 and the SQLiteDB in phase 2. Next to that the DatabaseHandler can perform CRUD operations on the database to create, read, update and delete DogProfiles. Similar to the DogProfile the read-profile operation does not find an object with the ID it will raise an exception. The operations and corresponding signatures for the DatabaseHandler are displayed in Table 3.

Operation	Signature
initialize-db-handler	-> DatabaseHandler
create-profile!	DatabaseHandler, DogProfile -> void
read-profile	DatabaseHandler, ID -> DogProfile
update-profile!	DatabaseHandler, DogProfile -> void
delete-profile!	DatabaseHandler, ID -> void

Table 3: ADT of the DatabaseHandler

Name: Arne Lieten

Email: Arne.Lieten@vub.be

Student Number: 0642644

VRIJE UNIVERSITEIT BRUSSEL

ProcessOrchestrator

Coordinates profile creation for all clients and holds a reference to the shared DatabaseHandler. It receives ProfileFields and calls the constructor in DogProfile. If the construction succeeds, the profile is stored in the InMemoryDatabase via the DatabaseHandler, otherwise an exception is raised. The signatures and operations for the ProcessOrchestrator are seen in Table 4.

Operation	Signature
initialize-process-orchestrator	DatabaseHandler -> ProcessOrchestrator
create-dog-profile!	ProcessOrchestrator, ProfileFields -> void

Table 4: ADT of the ProcessOrchestrator

Client ADTs

ClientOrchestrator

Represents one client session. Holds its own MainWindow and a reference to the shared ProcessOrchestrator. The connection to the MainWindow is to interact with the GUI, the reference to the ProcessOrchestrator is meant to connect to the server in phase 1 and will later be removed in phase 2 when the communication will go over TCP. The operations and signatures of the ClientOrchestrator are seen in Table 5.

Operation	Signature
initialize-client-orchestrator	ProcessOrchestrator, MainWindow -> ClientOrchestrator
submit-dog-profile!	ClientOrchestrator, RawFields -> void

Table 5: ADT of the ClientOrchestrator

MainWindow

Manages one client's GUI. This includes the fields to create a profile for a dog, a submit button and a feedback display on whether the submission was successful or not. When the submit button is clicked, it reads the field values and passes them to the registered callback. MainWindow also provides operations to open and close the window and display submission results.

Widget creation, layout and presentation remain inside MainWindow for phase 1. In later phases there can be a subdivision into more ADTs for the different widgets if required. The MainWindows' operations and signatures are found in Table 6.

Name: Arne Lieten

Email: Arne.Lieten@vub.be

Student Number: 0642644

VRIJE UNIVERSITEIT BRUSSEL

Operation	Signature
make-main-window	-> MainWindow
open-window!	MainWindow -> void
close-window!	MainWindow -> void
set-submit-callback!	MainWindow, SubmitCallback -> void
show-submit-result!	MainWindow, string, boolean -> void

Table 6: ADT of the MainWindow

For each client, startup creates a MainWindow and passes it, together with the shared ProcessOrchestrator to initialize the ClientOrchestrator. During initialization, ClientOrchestrator calls MainWindow's set-submit-callback! with a function that forwards RawFields to its own submit-dog-profile! operation. Startup then opens the window using `open-window!`. This choice was made so that the callback is connected before the user can submit the form.

Testing

The unit test section will be included after the feedback session with the assistants.